ESP32-S2 · ADS131M04 · FIRMWARE MAP

OzoneTrckr — Firmware Flow

How a measurement run executes, from cold boot through the loop to deep sleep — and the three independent ways the data gets off the device.

SETUP() runs once on power-up / reset

Wake the hardware safely

The board boots and powers up the analog front-end in the right order — analog off, then a cold start — before anything uses the SPI bus the ADC and screen share, so the ADC never desyncs. `setup()`

Turn on the screen, centre the filter

The TFT lights up (black at first) and the filter servo eases to its middle position. `displayInit()`

Find the sun

TRACKER

The four light sensors steer the two mount servos until all quadrants are equally bright — i.e. pointed at the sun. `sunflowerFindSun()`

Start the clock chip

Bring up the battery-backed real-time clock on the I²C bus; it holds the time even between runs. `rtc.begin()`

Mount the flash storage

FLASH

Open the on-chip filesystem *without* auto-formatting, so a damaged-but-readable previous run still gets a chance to be rescued below. `storageBegin()`

TFT Flash backup ready

Join WiFi and announce the device

WIFI

Connect with a fixed address, advertise the name **`ozone.local`**, and open a command port so a run can be ended remotely. `commandLinkBegin()`

TFT IP 192.168.1.50

Set the exact time

NTP

Ask an internet time server for the precise UTC and update the clock; if that fails, just trust the clock chip. `ntpSyncUTC()`

`TFT` `Clock: NTP synced`

Rescue the previous run

SERIAL + WIFI

If an earlier run is still on flash, send it out over serial **and** WiFi before it gets overwritten — so a missed upload never loses data. `uploadLastRun()` · `dumpLastRunToSerial()`

`TFT` `Last run sent (WiFi)`

Check the flash can be written

FLASH

Only now — after rescue — write-test the filesystem and reformat it only if it is genuinely corrupt. `storageEnsureWritable()`

Bring up the sensors

Read the weather sensor, load the saved site coordinates (the GPS stays asleep), and configure the ADC and its data-ready interrupt. `setup_ADC_CARD()`

Zero the ADC (dark calibration)

With no light in, measure each channel's offset and noise and load it into the ADC; the values are saved in the file header. `offsetCalibration()`

`TFT` `OFFSET CAL · Ch1 / Ch2`

Open the run file

FLASH

Create the rolling CSV, write the header, and confirm the write landed; from here every block is saved as it is measured. `runFileBegin()` → `/lastrun.csv`

`TFT` `Logging to flash`

THE RUN repeats ~every 32 s · 20 readings per loop

Keep WiFi alive

WIFI

Each loop, reconnect at once if the link dropped and nudge it so the router can't quietly kick the device off. `wifiKeepalive()`

Re-aim and work out where the sun is

TRACKER

Every 5th loop the mount re-centres on the sun; every loop the sun's elevation and azimuth are computed from the clock and coordinates. `sunflowerFindSun()` · `sunPosition()`

Measure through filter 1

Ease the filter to its first position, then take 10 readings — each one measures both channels, saves a row to flash + serial, and refreshes the live screen. `filter_rotation()` · `measurement()` · `storeMeasurement()`

Measure through filter 2

Swap to the second filter position and take 10 more the same way. A stop request never cuts a phase short — you always get a full 10 + 10.

```
TFT 17:42:05 28/06/26 · Ch1 612.34 Ch2 588.10 — the live dashboard, redrawn every reading
```

↻ repeat the loop…

…until any end condition:

- the planned number of readings is reached, or
- the **BOOT button** is pressed (works with no WiFi), or
- a **`stop`/`dump`** command arrives over WiFi.

END OF RUN deliver the data, then sleep

Close the run file

The whole run is already on flash (saved one block at a time), so this just flushes and closes it. `runFileEnd()`

① Serial

Print the CSV to a USB terminal if one is attached — works with no WiFi. `dumpLastRunToSerial()`

② WiFi → PC

Send the file to the PC's listener; keep retrying for a while (BOOT skips the wait). `uploadLastRun()`

③ Flash (kept)

Stays saved on flash and is re-sent on the next power-up.

```
TFT  Sending data… → Upload done
```

Park and power down

Return the filter, put the ADC to sleep, cut analog power, cleanly unmount flash, and hold the servo pins steady so they don't twitch. `storageEnd()`

SLEEP

Sleep until the next placement

The board switches fully off with no wake timer; you power-cycle to start the next run — which also re-sends this one if its upload was missed. `esp_deep_sleep_start()`

```
TFT  (display off)
```

DATA RESILIENCE — 3 CHANNELS

- F **Flash** — every block, to `/lastrun.csv`. Survives WiFi loss & battery death.
- S **Serial** — live `[…]` rows during the run, full CSV at end + next boot.
- W **WiFi** — TCP upload to the PC's `ncat -l 5000`.

TFT STATUS DOTS

- W **WiFi:** green = good, orange **w** = weak, red **x** = down. RSSI shown below.
- F **Flash:** green = logging & last write OK; red = off / write failed (live, per block).

NEOPIXEL STATUS (NO-TFT BUILD)

- **Per block:** green = flash + WiFi OK · orange = WiFi down (on flash) · red = write failed.
- **Setup:** blue = WiFi connecting · cyan = IP/link · green = NTP synced · orange = build-time fallback.
- **Actions:** magenta = sending data · green/orange = upload done / kept on flash.

COLOUR = SUBSYSTEM

- setup / general
- measurement & tracker
- WiFi / NTP / TCP
- flash storage (FFat)
- serial / delivery

ADC ADS131M04 — setup, DRDY handshake, and the sampling hierarchy

Setup — `setup_ADC_CARD()`

Device	**ADS131M04 — 24-bit, 4-ch simultaneous ΔΣ, SPI (bus shared with TFT)**
Data rate	**~250 SPS · High-Res mode · OSR 16384 (lowest noise)**
Channels	**CH0 + CH1 stored (CH2 also enabled); differential AINxP/AINxN**
PGA gain	**1× on CH0 / CH1 / CH2**
Full scale	**FS = 2^{23} = 8 388 608 counts, scaled to PREF = 1200 mV**
Bring-up	**`RESET` → `STANDBY` → write CLOCK/CFG/THRSHLD → `WAKEUP` → one flush read**

DRDY handshake — GPIO6, falling-edge IRQ

DRDY drops LOW when a conversion is ready and **stays low until the frame is clocked out over SPI** — it is *not* a free-running pulse in this configuration. The ISR sets a `volatile` flag; the sampling loop sees it, clears it, and calls `readADC()`, which reads the 24-bit words and releases DRDY for the next conversion.

So every conversion must be read — an interrupt-only wait with no read would deadlock. A 2 s no-DRDY timeout is the safety net (it flags "analog board disconnected").

Sampling hierarchy — how one stored row is built

Conversion
~4 ms · 250 SPS
hardware OSR 16384

×15 avg ▸

Sub-sample
~60 ms
low-noise, in mV

×20 Welford ▸

Block = 1 row
~1.2 s
mean ± σ → flash

×10 ▸

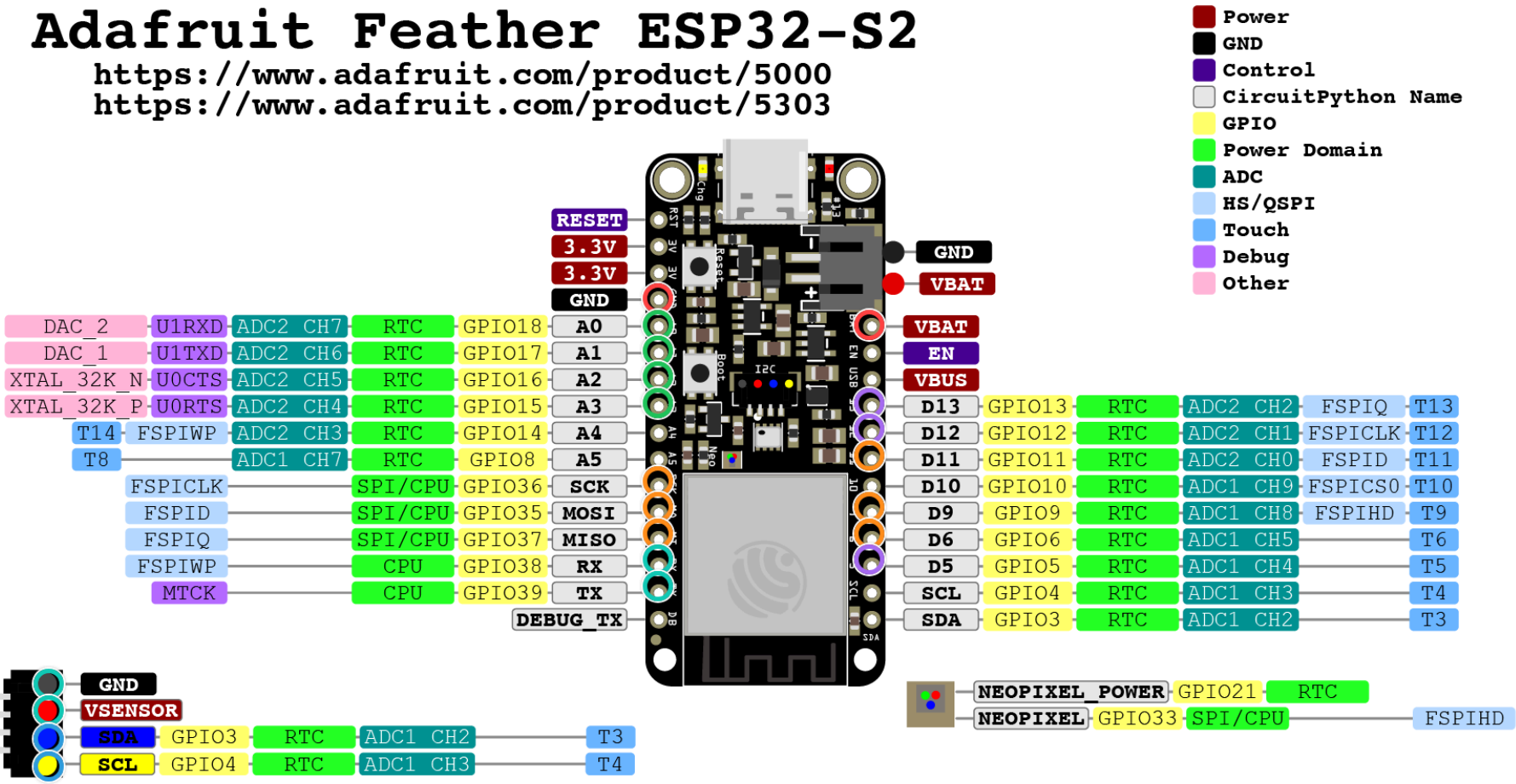
Phase
one filter position
(pos1 / pos2)

×2 ▸

Loop
20 rows
~32 s

Counts → mV: `V = (Σcounts / n) / FS × 1200 mV`, accumulated in `double`; mean and σ computed with Welford's algorithm for numerical stability. **Settling guard:** after every `CMD_WAKEUP` (i.e. each filter move) the first **25 conversions (~100 ms) are discarded** so the wake transient never lands in a block. **Offset cal:** 100 zero-input sub-samples are averaged into the ADC's hardware offset register and reported as mean ± σ in the CSV header. **Quiet windows:** the ADC is put to `STANDBY` during every filter-servo move, so servo current spikes and SPI traffic never coincide with sampling.

PINS Feather ESP32-S2 header pins used by the instrument



Stemma QT Power pin VSENSOR and I2C pull-up resistors are enabled/disabled by GPIO 7.
If you have a rev B board, set IO7 LOW to enable.
If you have a rev C board, set IO7 HIGH to enable.

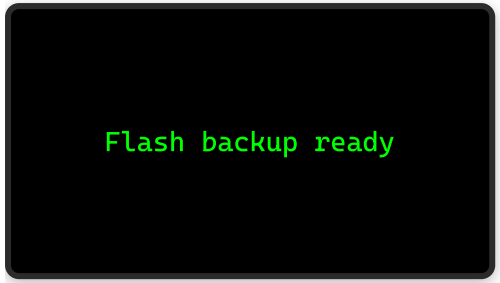
ADC board — SPI + control · Servos · LDR sun sensors · I²C sensors (STEMMA QT) · GPS — UART + power · Analog-board power

PINS GPIO assignments — what each marked pin does

PIN	GPIO	FUNCTION	SUBSYSTEM
SCK	36	ADC SPI clock (shared with TFT)	ADC board
MOSI	35	ADC SPI MOSI	ADC board
MISO	37	ADC SPI MISO	ADC board
D11	11	analog-board power enable (EN)	ADC board
D9	9	ADC chip-select (CS)	ADC board
D6	6	ADC data-ready (DRDY, interrupt)	ADC board
D13	13	tracker servo — vertical (elevation)	Servos
D12	12	tracker servo — horizontal (azimuth)	Servos
D5	5	filter servo (pos1 / pos2 / cal)	Servos
A0	18	LDR bottom-left	Sun sensors
A1	17	LDR top-right	Sun sensors
A2	16	LDR bottom-right	Sun sensors
A3	15	LDR top-left	Sun sensors
SDA	3	I²C data — DS3231 RTC + BME280 (via STEMMA QT)	I²C sensors
SCL	4	I²C clock — DS3231 RTC + BME280 (via STEMMA QT)	I²C sensors
RX	38	GPS UART RX (module TX → Feather)	GPS
TX	39	GPS UART TX (Feather → module RX)	GPS
VSENSOR	–	GPS + I²C-sensor power (STEMMA QT)	GPS
VBAT	–	analog-board supply (+)	Power
GND	–	analog-board ground	Power

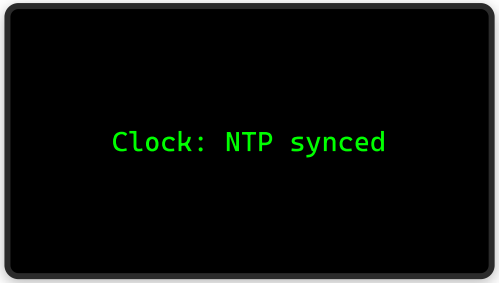
Not a header pin, so not marked above: **GPIO0** = the onboard **BOOT** button (WiFi-free early-stop). The DS3231 RTC and BME280 share the **STEMMA QT** connector (SDA/SCL + VSENSOR/GND); the GPS taps the same connector's **VSENSOR/GND** for power but talks over the header **RX/TX** (GPIO38/39) — it is held in backup/asleep this build, so its UART is idle and coordinates are hardcoded. The **analog board** is powered from **VBAT + GND**; the three servos run off a separate 5 V supply that shares ground with the Feather.

TFT what the 240×135 ST7789 shows, in run order

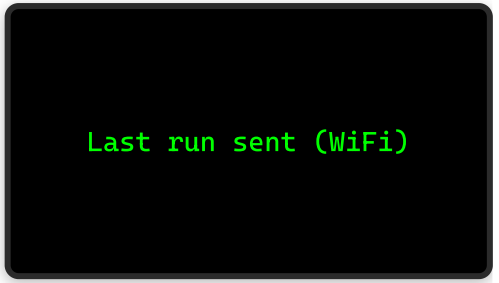


Setup · flash — FFat mounted (red `FLASH MOUNT FAILED` if not).

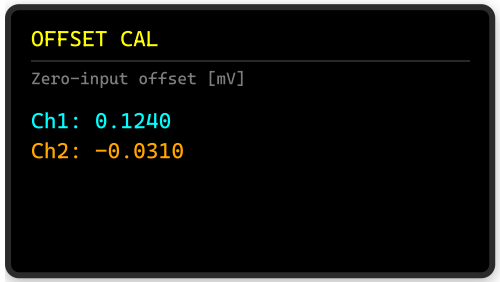
Setup · wifi — device address, no serial needed (also `ozone.local`).



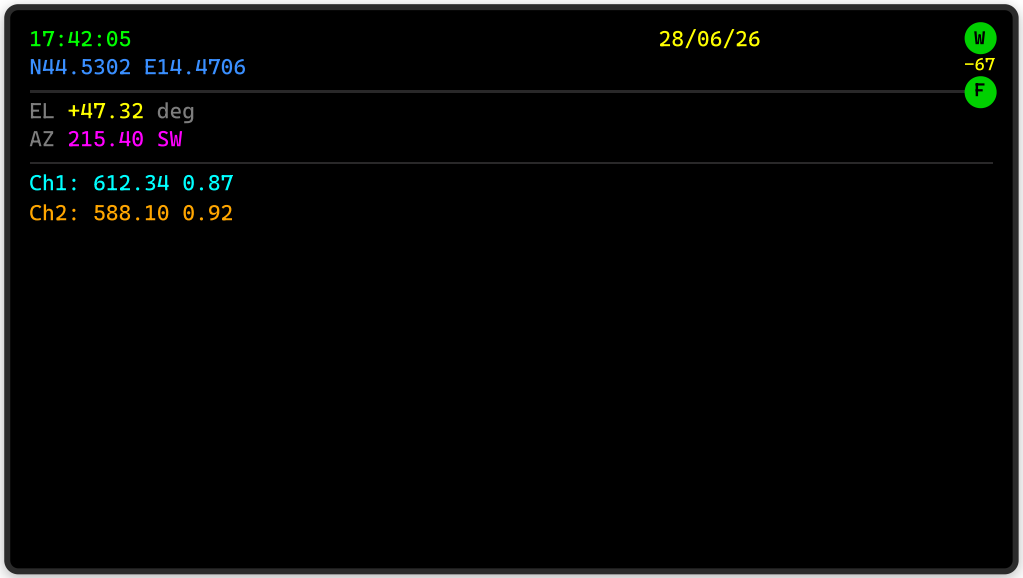
Setup · clock — or `Clock: RTC kept` / amber `Clock: build-time`.



Recovery — previous run re-delivered over serial + WiFi before it's overwritten.



Offset cal — per-channel dark offset, written into the CSV header.



Live dashboard — redrawn every block (flicker-free, in-place). Top-right: WiFi dot + RSSI (orange **w** weak / red **x** down), flash-health dot. Elevation is colour-coded by sun height.

End of run — then `Upload done` (green) or `On flash + serial`; the panel blanks (`displayOff()`) before deep sleep.

NEOPIXEL no-TFT build — what the single status LED shows, in run order

Setup checkpoints — each a short blink

Booting
dim white — powered on, cold-booting the ADC.

Flash OK
green ×2 — FFat mounted (red ×3 = mount/format failed).

WiFi connecting
solid blue — joining the network.

WiFi + IP
cyan ×2 — connected, address acquired (red ×3 = no WiFi).

Clock synced
green-teal ×2 — time set from NTP.

Clock fallback
orange ×3 — NTP failed → build-time clock.

Logging
green ×2 — run file open on flash (red = serial only).

During the run — one blink per block

All good
green — flash write OK and WiFi up.

WiFi down
orange — link lost, but the block is safely on flash.

Flash failed
red — the block's flash write did not commit.

Delivery — end of run

Sending data
magenta — uploading the run over WiFi (blinks per retry).

Upload done
green ×3 — the PC received the file.

Kept on flash
orange ×3 — no PC listener; data safe on flash + serial.

HOST commands you run on the PC — Windows & macOS

TASK	WIN WINDOWS — POWERSHELL	MAC MACOS — TERMINAL (ZSH)
Receive a run start the listener first, then end the run	`ncat -l 5000 > data.csv`	`nc -l 5000 > data.csv`
Reach the device	`ping ozone.local`	`ping ozone.local` `dns-sd -G v4 ozone.local`
End run early + send connect, then type `send` (or `stop`)	`ncat ozone.local 5050`	`nc ozone.local 5050`
Build & flash firmware	`pio run -t upload`	`pio run -t upload`
Erase flash enter download mode, close PuTTY first	`pio run -t erase`	`pio run -t erase`

Windows needs **Nmap** (for `ncat`) and **Apple Bonjour** to resolve `.local` names; macOS has `nc` and mDNS built in. The device connects out to `SECRET_PC_HOST:5000`, so the PC listener must be up *before* the run ends; the remote-stop server lives on the device at `:5050`. If `pio` isn't on PATH in PowerShell: `& "$env:USERPROFILE\.platformio\penv\Scripts\platformio.exe" run -t upload`.

OzoneTrckr · ESP32-S2 Feather + ADS131M04 · single rolling file `/lastrun.csv`, per-block commit, NTP clock, mDNS `ozone.local`, flicker-free TFT, latched servo pins on sleep. Working prototype — 2026-06-27.